



回溯法

—An Efficient Searching Algorithm

朱容波

Email: rongbozhu@126.com

Office: 一综B323

华中农业大学信息学院

本章内容



- Backtracking.回溯法
 - 3着色问题
 - 8皇后问题
 - 一般回溯方法
 - 子集和数问题
 - 哈密顿回路问题
 - 0-1背包问题

华中农业大学信息学院



13.1 简介

- 在许多真实世界的问题中，像大多数NP难问题一样，通过穷尽搜索数量巨大但有限多个可能性，可以获得一个解。
- 事实上对于所有这些问题都不存在用穷尽搜索之外的方法来解决问题的算法。



怎么样开发出系统化的搜索技术？

怎样能够将搜索空间减少到尽可能的小？

华中农业大学信息学院



回溯法的基本思想

(1) 解空间

设问题的解向量为 $X=(x_1, x_2, \dots, x_n)$ ， x_i 的取值范围为有穷集 S_i 。把 x_i 的所有可能取值组合，称为问题的解空间。每一个组合是问题的一个可能解。

华中农业大学信息学院



回溯法的基本思想

(1) 解空间

例：0/1背包问题， $S=\{0,1\}$ ，当 $n=3$ 时，0/1背包问题的解空间是：

$\{(0,0,0),(0,0,1),(0,1,0),(0,1,1),(1,0,0),(1,0,1),(1,1,0),(1,1,1)\}$

当输入规模为 n 时，有 2^n 种可能的解。

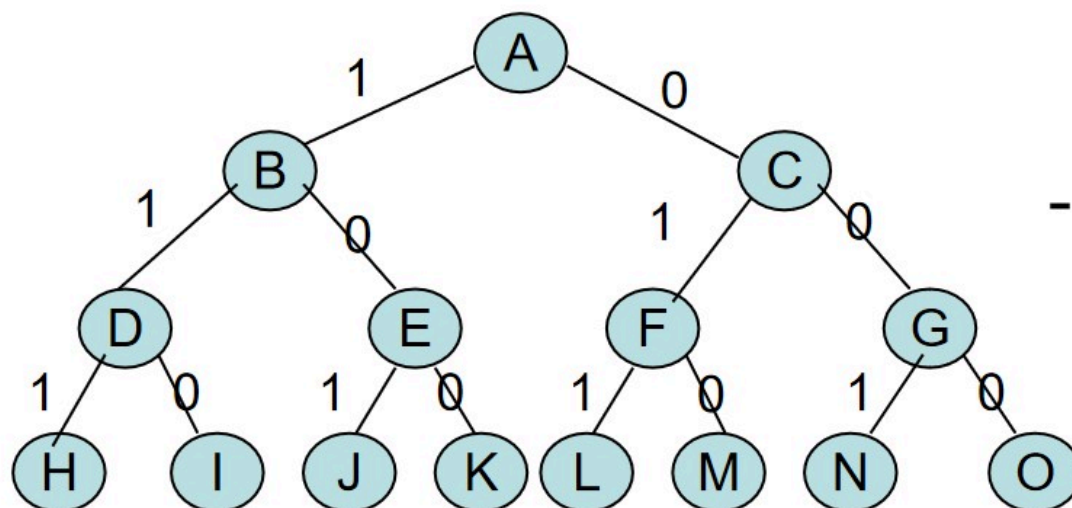
华中农业大学信息学院



回溯法的基本思想

(2) 状态空间树：问题解空间的树形式表示

例：n=3, 0/1背包问题的状态空间树。



----子集树

解空间大小: 2^n

华中农业大学信息学院



回溯法的基本思想

(1) 解空间

例：货郎担(旅行商)问题， $S=\{1,2,\dots,n\}$ ，当 $n=3$ 时， $S=\{1,2,3\}$ ，货郎担问题的解空间是：

$\{(1,1,1),(1,1,2),(1,1,3),(1,2,1),(1,2,2),(1,2,3),\dots,(3,3,1),(3,3,2),(3,3,3)\}$

当输入规模为 n 时，它有 n^n 种可能的解。

考虑到约束方程 $x_i \neq x_j$ 。因此，货郎担问题的解空间压缩为：

$\{(1,2,3),(1,3,2),(2,1,3),(2,3,1),(3,1,2),(3,2,1)\}$

当输入规模为 n 时，它有 $n!$ 种可能的解。

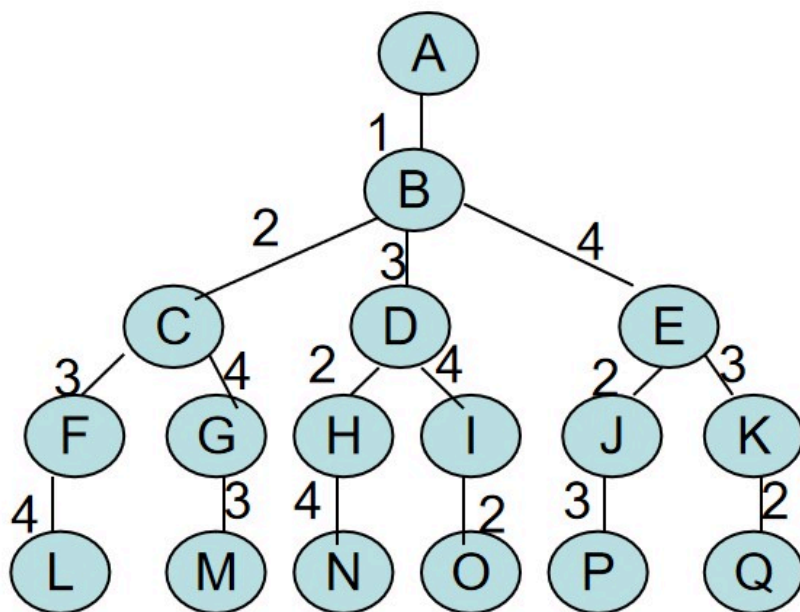
华中农业大学信息学院



回溯法的基本思想

(2) 状态空间树：问题解空间的树形式表示

例：n=4，旅行商问题的状态空间树。



----排列树
解空间大小: $n!$

华中农业大学信息学院



解空间树结构

- 树中的每个结点确定所求解问题的一个**问题状态**。
- 由根结点到其它结点的所有路径确定了这个问题的**状态空间**。
- **解状态**是这样一些问题状态 S ，对于这些问题状态,由根到 S 的那条路径确定了这解空间中的一个元组。
- **答案状态**是这样的一些解状态 S ，对于这些解状态而言，由根到 S 的这条路径确定了这问题的一个解。
- 解空间的树结构成为**状态空间树**。

华中农业大学信息学院



状态空间树=>问题解

- 对于任何一个问题,一旦设想出一种状态空间树,就可以系统地生成问题状态接着确定这些问题状态中的哪些状态是解状态,最后确定哪些解状态是答案状态,从而将问题解出。
- 问题状态的生成:从根结点开始生成其它结点。
- **活结点**:如果已生成一个结点而它的儿子结点还没有全部生成,则这个结点叫做活结点;当前正在生成其儿子结点的活结点叫E-结点(**正在扩展的结点**)。
- **死结点**:不再进一步扩展或者其儿子结点已经全部生成的生成结点是死结点。

华中农业大学信息学院



回溯法的问题要求

- 问题要求:

- (1)所要求的解必须能表示成一个 n -元组 $(x_1, x_2, \dots, x_n)$, 其中 x_i 是取自某个有穷集 S_i 。通常, 所求解的问题需要求取一个使某一个规范函数 $P(x_1, x_2, \dots, x_n)$ 取极值或满足规范条件的向量, 有时还要找出满足 P 的所有向量。
- (2)许多问题还要求所有的解满足一组综合的约束条件, 这些条件分两种类型: 显式约束和隐式约束。
 - 显式约束限定每个 x 只从一个给定的集合上取值, 满足显式约束的所有元组确定一个可能的解空间;
 - 隐式约束规定解空间中那些实际上满足规范函数的元组, 因此, 描述了 x_i 必须彼此相关的情况。

华中农业大学信息学院



回溯法的求解方法

- 问题的求解方法:

假定集合 S_i 的大小为 m_i , 于是就有 $m = m_1 m_2 \dots m_n$ 个 n -元组可能满足函数 P 。

- 问题的硬性处理求解方法:

构造出 m 个 n -元组并逐一测试它们是否满足 P , 从而找出该问题的所有最优解。

- 问题的回溯处理求解方法:

不断地用修改过的规范函数 $P_i(x_1, x_2, \dots, x_n)$ (有时称为限界函数) 去测试正在构造中的 n -元组的部分向量 $(x_1, x_2, \dots, x_i)$, 看其是否可能导致 (最优) 解。如果判定 $(x_1, x_2, \dots, x_i)$ 不可能导致 (最优) 解, 那么就将可能要测试的 $m_{i+1} m_{i+2} \dots m_n$ 个向量一概略去。

回溯法比硬性处理作的测试 (m 次) 要少得多。

华中农业大学信息学院



问题举例

- 8—皇后问题：在一个 8×8 的棋盘上放8个皇后，且使得每两个之间都不能互相“攻击”，即都不出现在同一行、同一列及同一条斜对角线上。
 - 解的表示：假定第 i 个皇后放在第 i 行，用8—元组 $(x_1, x_2, \dots, x_8)$ 表示解，其中 x_i 表示放置皇后 i 的列号。
 - 显示约束： $S_i = (1, 2, 3, \dots, 8), 1 \leq i \leq 8$
 - 隐示约束：没有两个 $x_i (1 \leq i \leq 8)$ 可以相同（在不同的列上），且没有两个皇后在同一斜对角线上。

华中农业大学信息学院



回溯法的定义

- **回溯法** (backtracking) 是一种系统地搜索问题解的方法。为实现回溯，首先需要定义一个**解空间** (solution space)，然后以**易于搜索**的方式组织解空间，最后用**深度优先**的方法搜索解空间，获得问题的解。

华中农业大学信息学院



回溯法的步骤

- 回溯法求解的三个步骤：
 - (1) 定义一个解空间，它包含问题的解
 - (2) 用易于搜索的方式组织解空间
 - (3) 深度优先搜索解空间，获得问题的解。

华中农业大学信息学院



深度优先策略

- 确定了解空间的组织结构后，回溯法就从开始顶点（解空间树的根顶点）出发，以深度优先的方式搜索整个解空间。这个开始顶点就成为一个活顶点，同时也成为当前的扩展顶点。在当前的扩展顶点处，搜索向纵深方向移至一个新顶点。这个新顶点就成为一个新的活顶点，并且成为当前的扩展顶点。

华中农业大学信息学院



深度优先策略

- 如果在当前的扩展顶点处不能再向纵深方向移动，则当前的扩展顶点就成为死顶点。此时应往回移动（回溯）至最近一个活顶点处，并使这个活顶点成为当前扩展顶点。回溯法即以这种工作方式递归地在解空间中搜索，直至找到要求的解或解空间中已无活结点时为止。
- 回溯法实际上在搜索的同时逐步生成解空间树（实际只需要一部分）。也就是说，对于实际问题我们不必要搜索整个解空间。

华中农业大学信息学院



深度优先策略

- 为了使搜索更加有效，我们常常在搜索过程中加一些判断以决定搜索是否该终止或改变路线。通常采用两种策略来避免无效的搜索，提高回溯法的搜索效率。其一是使用**约束函数**在扩展顶点处剪去不满足约束的子树；其二是用**限界函数**剪去不能得到最优解的子树。这两种函数统称为**剪枝函数**。

华中农业大学信息学院



13.2 3着色问题

? 问题描述

- 给出一个无向图 $G=(V,E)$ ，需要用三种颜色之一为 v 中的每个顶点着色，三种颜色分别为1,2和3，使得没有两个邻接的顶点有同样的颜色。
- 一种着色可以用 n 元组 $(c_1, c_2, \dots, c_n)$ 来表示，使 $c_i \in \{1, 2, 3\}$, $1 \leq i \leq n$.
- 例如， $(1,2,2,3,1)$ 表示一个有5个顶点的图的着色，一个 n 个顶点的图共有 3^n 种可能的着色(合法的和非法的)。

华中农业大学信息学院



搜索树

- 所有可能的着色的集合可以用一棵完全的三叉树来表示，称为**搜索树**。在这棵树中，从根到叶节点的每一条路径代表一种**着色指派**。

华中农业大学信息学院



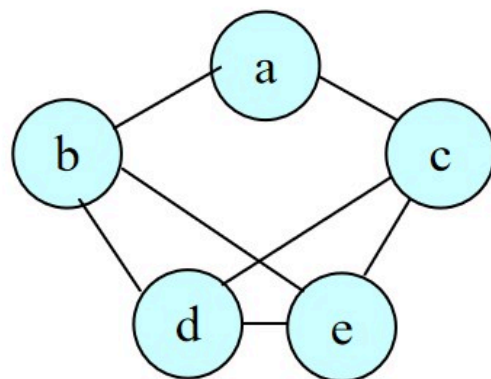
Backtracking: Partial Coloring Scheme

- 如果没有两个邻接的着色顶点有同样的颜色，图的一个不完全着色称为**部分解**。
- Idea:回溯法通过每次一个节点地生成基底树来工作。
 - 如果从根到当前节点的路径对应于一个合法着色过程就终止(除非期望找到不止一种的着色)
 - 如果这条路径的长度小于 n ，并且相应的着色是部分的，那么就生成现节点的一个子节点，并将它标记为**现节点**。
 - 另一方面，如果对应的路径不是部分的，那么现节点标识为**死节点**并生成对应于另一种颜色的**新节点**。
 - 如果所有三种颜色都已经试过且没有成功，搜索就回溯到父节点，它的颜色被改变，依次类推。

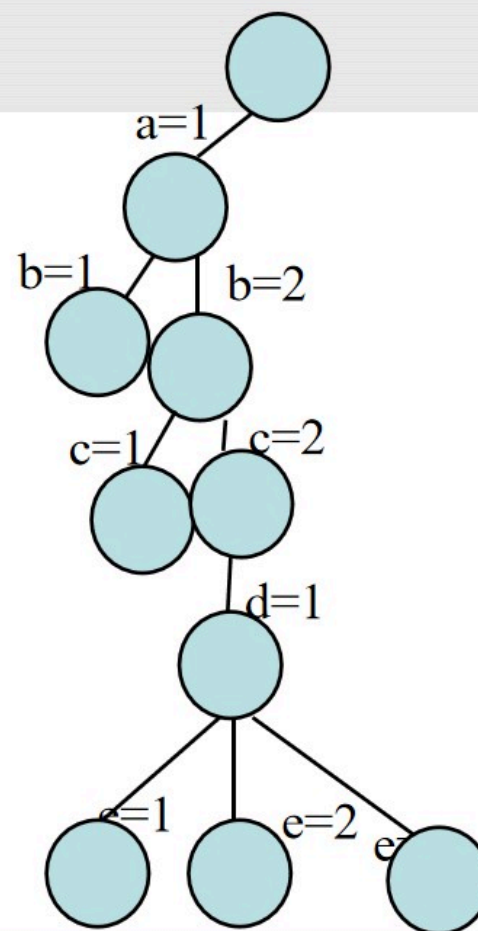
华中农业大学信息学院



An example



Find a solution!



华中农业大学信息学院



观察结论

1. 我们仅生成了由364个节点组成的搜索树中10个节点后就到达了解。
2. 节点是用**深度优先搜索**的方法生成的;
3. 不需要存储整棵搜索树, 我们只需要存储根到当前活动节点的路径。事实上, 根本没有生成有形的物理节点, 整棵树是隐含的。在上面的例子中, 我们只需要保存颜色指派的踪迹就可以了。

华中农业大学信息学院



算法 13.1 3-COLORREC (递归)

输入: 无向图 $G=(V,E)$.

输出: G 的顶点的3着色 $c[1..n]$, 其中每个 $c[j]$ 为1,2或3。

```
1. for  $k \leftarrow 1$  to  $n$ 
2.  $c[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $graphcolor(1)$ 
6. if  $flag$  then output  $c$ 
7. else output "no solution"
```

过程 $graphcolor(k)$

```
1. for  $color=1$  to  $3$ 
2.  $c[k] \leftarrow color$ 
3. if  $c$  为合法着色 then set  $flag \leftarrow true$  and exit
4. else if  $c$  是部分的 then  $graphcolor(k+1)$ 
5. end for
```

华中农业大学信息学院



算法13.2 3-COLORITER (迭代)

输入: 无向图 $G=(V,E)$.

输出: G 的顶点的3着色 $c[1..n]$, 其中每个 $c[i]$ 为1,2或3。

```
1. for  $k \leftarrow 1$  to  $n$ 
2.  $c[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $k \leftarrow 1$ 
6. while  $k \geq 1$ 
7. while  $c[k] \leq 2$ 
8.  $c[k] \leftarrow c[k] + 1$ 
9. if  $c$ 为合法着色, then 设置 $flag \leftarrow true$ 且从两个while循环退出
10. else if  $c$ 是部分解 then  $k \leftarrow k + 1$  {前进}
11. end while
12.  $c[k] \leftarrow 0$ 
13.  $k \leftarrow k - 1$  {回溯}
14. end while
15. if  $flag$  then output  $c$ 
16. else output "no solution"
```

华中农业大学信息学院



算法分析

- 对于这两种算法的时间复杂性，我们注意到在最坏情况下生成了 $O(3^n)$ 个节点。对于每个生成的节点，如果当前着色是合法的、部分的，或者二者都不是，这需要 $O(n)$ 的工作来检查。因此，在最坏情况下，全部的运行时间是 $O(n3^n)$ 。
- 空间复杂度是 $\Theta(n)$ 。

华中农业大学信息学院



课堂练习

- 用回溯法求下列不等式的所有整数解，要求给出伪码和解：

$$\begin{cases} 3x_1 + 4x_2 + 2x_3 \leq 12 \\ x_1, x_2, x_3 \text{ 为非负整数} \end{cases}$$

华中农业大学信息学院



13.3 8皇后问题

? 8皇后问题描述

- 如何在8x8的国际象棋棋盘上安排8个皇后，使得没有两个皇后能互相攻击？
- 也就是说：如果两个皇后处在同一行、同一列或同一条对角线上，则她们能互相攻击。
- 为了简化讨论，我们将研究4皇后问题。

华中农业大学信息学院



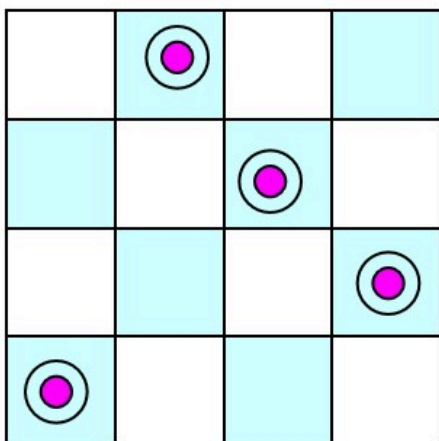
4皇后问题

- 考虑一个4x4的棋盘，由于没有两个皇后能处在同一行，所以每个皇后都在不同的行上。又因为每行有4个位置，就有 4^4 种可能的布局，每种可能的布局可以用一个有4个分量的向量 $x=(x_1, x_2, x_3, x_4)$ 来描述。
- 实际上，因为没有两个皇后能放在同一列上，一个合法的放置对应于数1, 2, 3, 4的一个排列，这把搜索空间从 4^4 减少到 $4!$ 。

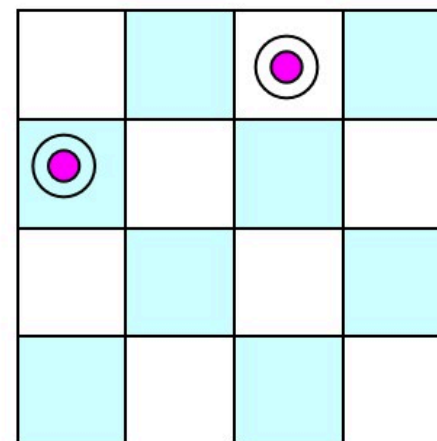
华中农业大学信息学院



例子



The vector $(2,3,4,1)$



The partial vector $(3,1)$

华中农业大学信息学院



互相攻击

假设：问题的解由 $(x_1, \dots, x_n)$ 表示

- 因为每行只放一个皇后，两个皇后不能处在同一行上。
- 如果两个皇后处在同一列上，他们互相攻击
 - In this case, $x_i = x_j$;
- 如果两个皇后处在同一条对角线上，他们互相攻击
 - In this case, $x_i - x_j = i - j$ or $x_i - x_j = j - i$.

华中农业大学信息学院



算法13.3 4-QUEENS(迭代)

输入: 空。

输出: 对应于4皇后问题的解的向量 $x[1..4]$ 。

```
1. for  $k \leftarrow 1$  to 4
2.  $x[k] \leftarrow 0$  {没有皇后放置在棋盘上}
3. end for
4. flag  $\leftarrow$  false
5.  $k \leftarrow 1$ 
6. while  $k \geq 1$ 
7.   while  $x[k] \leq 3$ 
8.      $x[k] \leftarrow x[k] + 1$ 
9.     if  $x$  为合法解 then set flag  $\leftarrow$  true 且从两个while循环退出
10.    else if  $x$  是部分解 then  $k \leftarrow k + 1$  {前进}
11.  end while
12.  $x[k] \leftarrow 0$ 
13.  $k \leftarrow k - 1$  {回溯}
14. end while
15. if flag then output  $x$ 
16. else output “no solution”
```

华中农业大学信息学院



例子：4-Queens Problem



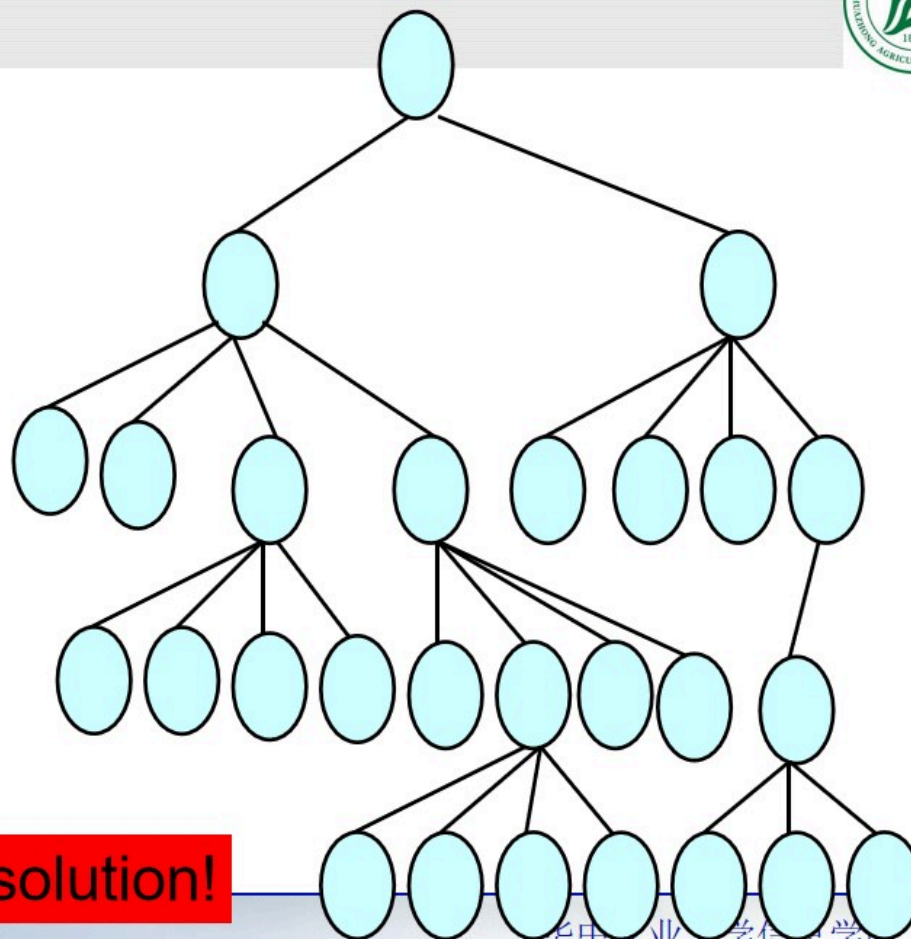
1			
X	X	2	
X	X	X	X

1			
			2
X	3		
X	X	X	X

1			
			2
		X	X

	1		
X	X	X	2
3			
X	X	4	

Find a solution!



华中农业大学信息学院



算法分析

- 现在考虑一个求解一般的 n 皇后问题的蛮力方法
 - 由于没有两个皇后能放在同一列上，则解向量必须是数 $1, 2, \dots, n$ 的一个排列。这样，蛮力方法可以改进为测试 $n!$ 种布局而不是 n^n 种。
 - 考虑 $(n-2)!$ 个向量，它们对应于前两个皇后放在第一列的那些布局，蛮力方法要盲目地测试所有这些向量，而在回溯法中用 $O(1)$ 次测试就可以避免这些测试。尽管回溯法在最坏情况下要用 $O(n^n)$ 时间来求解 n 皇后问题，根据经验它在有效性上远远超过蛮力方法的 $O(n!)$ 时间，作为它的可期望运行时间通常要快得多。

华中农业大学信息学院



课堂练习

- 最小重量机器设计问题。某设备需要4种配件，每种1件。有3个供应商提供这些配件。表给出相关的价格和每种配件的重量。从中选择这4种配件，使得总价值不超过120，总重量最轻。

零件编号	供应商1		供应商2		供应商3	
	价格	重量	价格	重量	价格	重量
1	10	5	8	6	12	4
2	20	8	21	10	30	5
3	40	5	42	4	30	10
4	30	20	60	10	45	15

华中农业大学信息学院



13.4 一般回溯方法

- Representation of solution (解的表现形式)
 - 一般回溯算法，它可以作为一种系统的搜索方法应用到一类搜索问题当中，这类问题的解由满足事先定义好的某个约束的向量 $(x_1, x_2, \dots, x_i)$ 组成。这里 i 是0到 n 之间的某个整数，其中 n 是一个取决于问题阐述的常量。在已经提到的两种算法——3着色和8皇后问题中， i 是固定不变的。
 - 然而在一些问题中， i 可以像下面的例子展示的那样，对于不同的解可能有所不同。

华中农业大学信息学院



例子: fixed and variable length of solution

- 给定一个n个整数的集合 $X = \{x_1, x_2, \dots, x_n\}$ 和整数y, 找出和等于y的X的子集Y。比如说, 如果 $X = \{10, 20, 30, 40, 80, 60\}$ and $y = 60$
- 变长解:
 - 三种不同长度的解:
 $\{10, 20, 30\}$
 $\{20, 40\}$
 $\{60\}$
- 定长解:
 - 这个问题可以用另一种方法明确表达, 解是一种明显的长度为n的布尔向量:
 $\{1, 1, 1, 0, 0, 0\} \{0, 1, 0, 1, 0, 0\} \{0, 0, 0, 0, 0, 1\}$. 我们要善于将变长解转换为定长解!

华中农业大学信息学院



回溯基本思想

- 在回溯法中，解向量中每个 x_i 都属于一个有限的线序集 X_i ；因此，回溯算法按词典序考虑笛卡儿积 $X_1 \times X_2 \times \dots \times X_n$ 的元素
- 算法最初从空向量开始，然后选择 X_1 中最小元素作为 x_1 ，如果 (x_1) 是一个部分解，算法通过从 X_2 中选择最小的元素作为 x_2 继续，如果 (x_1, x_2) 是一个部分解，那么就包括 X_3 中最小的元素。
- 一般地，假定算法已经检测到部分解为 $(x_1, x_2, \dots, x_j)$ ，它然后再去考虑向量 $v = (x_1, x_2, \dots, x_j, x_{j+1})$ 。我们有下面的情况：

华中农业大学信息学院



回溯基本思想(continue)

1. 如果 v 表示问题的最后解，算法记录下它作为一个解，在仅希望获得一个解时终止或者继续去找出其他解。
2. (向前步骤)。如果 v 表示一个部分解，算法通过选择集合 X_{j+2} 中的最小元素向前。
3. 如果 v 既不是最终的解，也不是部分解，则有两种子情况：
 - a) 如果从集合 X_{j+1} 中还有其他的元素可选择，算法将 x_{j+1} 置为 X_{j+1} 中的下一个
 - b) (回溯步骤)。如果从集合 X_{j+1} 中没有更多的元素可选择，算法通过将 x_j 置为 X_j 中的下一个元素回溯；如果从集合 X_j 中仍然没有其他的元素可以选择，算法通过将 x_{j-1} 置为 X_{j-1} 中的下一个元素回溯，依次类推。

华中农业大学信息学院



算法13.4 BACKTRACKREC

输入:集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出:解向量 $v=(x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $\text{advance}(1)$
4. **if** flag **then output** v
5. **else output** “no solution”

过程 $\text{advance}(k)$

1. **for** each $x \in X_k$
2. $x_k \leftarrow x$; 将 x_k 加入 v
3. **if** v 为最终解 **then** set $\text{flag} \leftarrow \text{true}$ and **exit**
4. **else if** v 是部分解 **then** $\text{advance}(k+1)$
5. **end for**

华中农业大学信息学院



算法13.5 BACKTRACKITER

输入:集合 $X_1, X_2, \dots, X_n$ 的清楚的或隐含的描述。

输出:解向量 $v=(x_1, x_2, \dots, x_i)$, $0 \leq i \leq n$.

1. $v \leftarrow ()$
2. $\text{flag} \leftarrow \text{false}$
3. $k \leftarrow 1$
4. **while** $k \geq 1$
5. **while** X_k 没有被穷举
6. $x_k \leftarrow X_k$ 中的下一个元素; 将 x_k 加入 v
7. **if** v 为最终解 **then** set $\text{flag} \leftarrow \text{true}$ 且两个while循环退出
8. **else if** v 是部分解 **then** $k \leftarrow k+1$ {前进}
9. **end while**
10. 重置 X_k , 使得下一个元素排在第一位
11. $k \leftarrow k-1$
12. **end while**
13. **if** flag **then output** v
14. **else output** "no solution"

华中农业大学信息学院



子集和数问题

- 已知 $n+1$ 个正数: $w_i, (1 \leq i \leq n)$, 和 M , 要求找出 w_i 的和数是 M 的所有子集
- 解的表示形式:
 - 采用定长元组: 解为 n 元组 $(X_1, X_2, \dots, X_n)$, 其中 X_i 表取0或1, 表示 w_i 的在子集合中是否出现。
- 约束条件: 并设定 $w_i (1 \leq i \leq n)$ 按非降次序排列。

$$\sum_{i=1}^n w(i)x(i) = M$$

华中农业大学信息学院



限界函数

- $B_k(x(1), \dots, x(k)) = \text{true}$ 当且仅当:

$$\sum_{i=1}^k w(i)x(i) + w(k+1) > M$$

- 假定 w_i 已经按照非降次序排列, 则可以强化限界函数, 如果下式成立, $B_k(x(1), \dots, x(k))$ 为假:

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \geq M$$

- 故, 假定 w_i 已经按照非降次序排列, 限界函数 $B_k(x(1), \dots, x(k)) = \text{true}$ 当且仅当:

$$\sum_{i=1}^k w(i)x(i) + \sum_{i=k+1}^n w(i) \geq M \text{ 并且 } \sum_{i=1}^k w(i)x(i) + w(k+1) \leq M$$

华中农业大学信息学院



算法SUMOFSUBREC

输入: $w[1,2,\dots,n]$ and M , $w[i]>0$, $M>0$.

输出: A solution vector $x[1,2,\dots,n]$, where each $x[i]$ is 0 or 1.

```
1. for  $k \leftarrow 1$  to  $n$ 
2.    $x[k] \leftarrow 0$ 
3. end for
4.  $flag \leftarrow false$ 
5.  $s \leftarrow 0$ ;  $r \leftarrow w[1]+w[2]+\dots+w[n]$ 
6. if  $r \geq M$  then SUBOFSUB( $s,1,r$ ) endif
7. if  $flag$  then output  $x$ 
8. else output "no solution"
9. end if
```

Procedure SUMOFSUB (s,k,r)

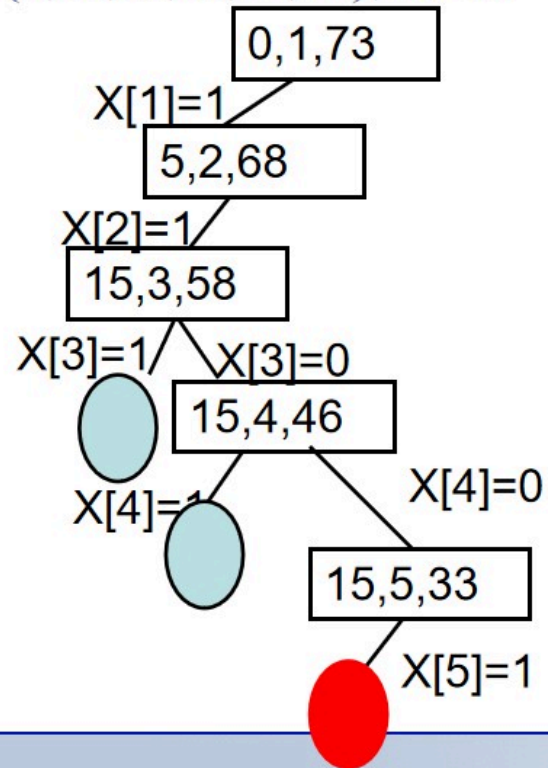
```
1.  $x[k] \leftarrow 1$ 
2. if  $s+w[k]=M$  //  $x$  是合法解
3. then set  $flag \leftarrow true$  and exit
4. else
5. if  $s+w[k]+w[k+1] \leq M$  //  $x$  是部分的  $B_k=true$ , 第  $k$  个数选入
6. then SUMOFSUB( $s+w[k],k+1,r-w[k]$ ) // 递归调用第  $k+1$  个数
7. end if
8. end if
9. if  $s+r-w[k] \geq M$  and  $s+w[k+1] \leq M$  //  $B_k=false$  第  $k$  个数不选入
10. then  $x[k] \leftarrow 0$ , SUMOFSUB( $s,k+1,r-w[k]$ )
11. end if
```

华中农业大学信息学院



Example

- $w[1..6] = \{5, 10, 12, 13, 15, 18\}$, $M = 30$



华中农业大学信息学院



课堂练习

- 一个四阶Latin方是一个4*4的方格，在它的每个方格内填入1,2,3,4，并使得每个数字在每行、每列都恰好出现一次。用回溯法求出所有第一行为1,2,3,4的Latin方。

1	2	3	4
3	4	1	2
4	3	2	1
2	1	4	3

对应解<3,4,1,2,4,3,2,1,2,1,4,3>

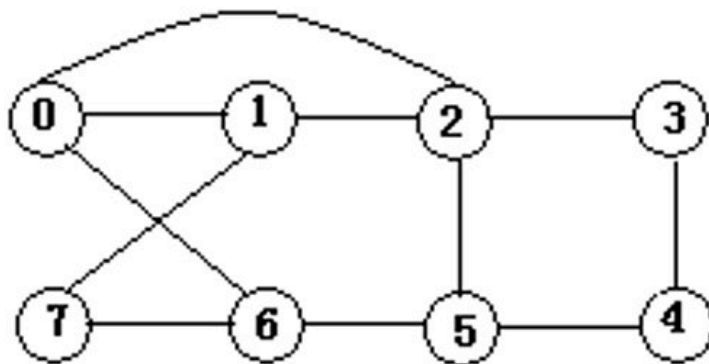
华中农业大学信息学院



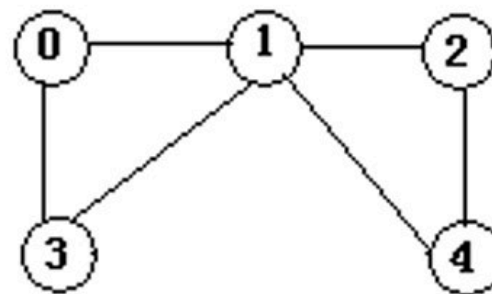
8.5 哈密顿回路问题

■ 已知图 $G=(V,E)$ 是一个 n 个结点的连通图。连通图 G 的一个**哈密顿环**(Hamiltonian cycle)是图 G 的一个回路，它经过图中每个结点，且只经过一次。一个哈密顿环是从某个结点 $v_0 \in V$ 开始，沿着图 G 的 n 条边环行的一条路径 $(v_0, v_1, \dots, v_{n-1}, v_n)$ ，其中， $(v_i, v_{i+1}) \in E, 0 \leq i < n$ ，它访问图中每个结点且只访问一次，最后返回开始结点，即除 $v_0 = v_n$ ，路径上其余结点各不相同。

华中农业大学信息学院



(a) 图 G_1



(b) 图 G_2

图8-15 图 G_1 包含哈密顿环, 图 G_2 不含哈密顿环

华中农业大学信息学院



8.5.2 哈密顿环算法

- 对于 n 个结点的图 $G=(V,E)$ 的哈密顿环问题, 可采用 n -元组表示问题的解 $(x_0, x_1, \dots, x_{n-1})$ 。每个 $x_i \in \{0, 1, \dots, n-1\}$, $0 \leq i < n$, 代表路径上一个结点的编号, 这就是显式约束。因此解空间的大小为 n^n 。其隐式约束可描述为: $x_i \neq x_j, 0 \leq i, j < n, i \neq j$, 且 $(x_i, x_{i+1}) \in E, x_i, x_{i+1} \in V, i=0, 1, \dots, n-2$, 又 $(x_{n-1}, x_0) \in E$ 。
- 哈密顿环问题的分析和求解方法与图的 m -着色问题非常相似

华中农业大学信息学院



● 【程序8—7】 哈密顿环算法

```
template <class T>
void MGraph<T>::NextValue(int k,int *x)
{
    do {
        x[k]=(x[k]+1)% n;
        if (!x[k]) return;
        if (a[x[k-1]][x[k]]) {
            for (int j=0;j<k;j++) if (x[j]==x[k]) break;
            if (j==k)
                if ((k<n-1)||((k==n-1) && a[x[n-1]][x[0]]))
                    return;
        }
    } while(1);
}
```

华中农业大学信息学院



```
template <class T>
void ExtMGraph<T>::Hamiltonian(int k,int *x)
{
    do {
        NextValue(k,x);
        if (!x[k]) return;
        if (k==n-1) {
            for (int i=0; i<n; i++) cout<<x[i]<< ' ';
            cout << "0\n";
        }
        else Hamiltonian(k+1,x);
    } while (1);
}
```

华中农业大学信息学院



```
template <class T>
void ExtMGraph<T>:: Hamiltonian(int *x)
{
    Hamiltonian(1,x);
}
```

华中农业大学信息学院



课堂练习

- 分派问题。给 n 个人分配 n 件工作，给第 i 个人分配第 j 件工作的成本是 $C(i,j)$ ，试求成本最小的工作分配方案。回溯法求解。

华中农业大学信息学院



8.6 0/1背包问题

回溯法：是一个既带有系统性又带有跳跃性的搜索算法。它在包含问题的所有解的解空间树中，按照深度优先的策略，从根结点出发搜索解空间树。算法搜索至解空间树的任一结点时，总是先判断该结点是否肯定不包含问题的解。如果肯定不包含，则跳过对以该结点为根的子树的系统搜索，逐层向其原先结点回溯。否则，进入该子树，继续按深度优先的策略进行搜索。

课堂上老师已经讲解过用回溯法解决n-皇后问题，m-图着色问题以及哈密顿环问题，他们有相同的特征即问题的求解目标都是求满足约束条件的全部可行解。而0/1背包是最优化问题，还需要使用限界函数剪去已能确认不含最优答案结点的子树。

华中农业大学信息学院



回溯法解决0/1背包问题

- 运用回溯法解题通常包含以下三个步骤：
 - a. 针对所给问题，定义问题的解空间；
 - b. 确定易于搜索的解空间结构；
 - c. 以深度优先的方式搜索解空间，并且在搜索过程中用剪枝函数避免无效搜索；

华中农业大学信息学院



0/1背包问题概述

在0/1背包问题中，需对容量为 c 的背包进行装载。从 n 个物品中选取装入背包的物品，每件物品 i 的重量为 w_i ，价值为 p_i 。对于可行的背包装载，背包中的物品的总重量不能超过背包的容量，最佳装载是指所装入的物品价值最高，即 $\sum_{i=1}^n p_i x_i$ 取得最大值。约束条件为 $\sum_{i=1}^n w_i x_i \leq C$ 和 $x_i \in [0,1] (1 \leq i \leq n)$ 。

在这个表达式中，需求出 x_i 的值。 $x_i=1$ 表示物品 i 装入背包中， $x_i=0$ 表示物品 i 不装入背包。

华中农业大学信息学院



回溯法解决01背包问题

- 01背包形式化描述:
- 给定 $M>0$, $w_i>0$, $p_i>0$ ($0\leq i<n$), 要求一个 n -元组 $(x_0, x_1, \dots, x_{n-1})$, $x_i \in \{0, 1\}$, $0\leq i<n$, 使得 $\sum_{i=0}^n w_i x_i \leq M$ 且 $\sum_{0\leq i<n} p_i x_i$ 最大。(x_i=1表示物品i装入背包中, x_i=0表示物品i不装入背包)

华中农业大学信息学院



回溯法解决01背包问题

- 判定可行解的约束条件是：
- $\sum_{i=0}^n w_i x_i \leq M$, $w_i > 0$, $x_i = 0$ 或者 1 ($0 \leq i < n$)
- 目标函数取最大值：
- $\max \sum_{i=0}^{n-1} p_i x_i$, $p_i > 0$, $x_i = 0$ 或 1 ($0 \leq i < n$)

华中农业大学信息学院



问题分析:

- 首先是将可供选择的物品的个数输入程序，将物品排成一列，计算总物品的体积 s ，然后输入背包的实际体积 V ，如果背包的体积小于0或者大于物品的总体积 s ，则判断输入的背包体积错误，否则开始顺序选取物品装入背包，假设已选取了前 i 件物品之后背包还没有装满，则继续选取第 $i+1$ 件物品，若该件物品“太大”不能装入，则弃之而继续选取下一件，直至背包装满为止。
- 但如果在剩余的物品中找不到合适的物品以填满背包，则说明“刚刚”装入背包的那件物品“不合适”，应将它取出“弃之一边”，继续再从“它之后”的物品中选取，如此重复，直至求得满足条件的解。(回溯)

华中农业大学信息学院



回溯法解决0/1背包问题

- 0/1背包问题是一个子集选取问题，适合于用子集树表示0/1背包问题的解空间。在搜索解空间树是，只要其左儿子节点是一个可行结点，搜索就进入左子树，在右子树中有可能包含最优解是才进入右子树搜索。否则将右子树剪去。

华中农业大学信息学院



更优的算法：利用贪心法最优值做上界

- 对于0-1背包问题回溯法的一个实例， $n=4$ ， $M=7$ ， $p=[9,10,7,4]$ ， $w=[3,5,2,1]$.这4个物品的单位重量价值分别为 $[3,2,3.5,4]$.以物品为单位价值的递减序装入物品。先装入物品4，然后装入物品3和1.装入这3个物品后，剩余的背包容量为1，只能装入0.2个物品2.由此可得到一个解为 $x=[1,0.2,1,1]$ ，其相应的价值为22.尽管这不是一个可行解，但可以证明其价值是最有大的上界。因此，对于这个实例，最优值不超过22.
- 还有没有更优的算法呢？

华中农业大学信息学院



更优的算法：最优解的动态更新和限界函数

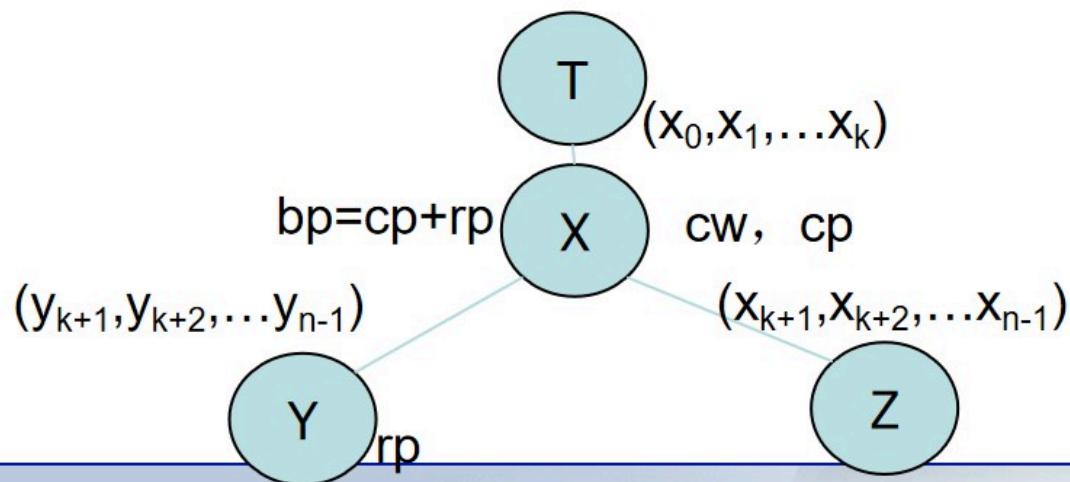
- L （当前最优值）始终为已搜索到的答案节点中受益的最大值。
- 计算右子树中解的上界的更好方法是将剩余物品依其单位重量价值排序，然后依次装入物品，直至装不下时，再装入该物品的一部分而装满背包。由此得到的价值是右子树中解的上界。
- 设 r 是当前剩余物品价值总和； cp 是当前结点 X 的价值； bp 是当前 X 结点上界函数值当 $cp+rp=bp < L$ 时可剪去以 X 为根的子树。

华中农业大学信息学院



限界函数图示

- L 始终为已搜索到的答案节点中受益的最大值，最优解必定大于等于 L ，对于任意结点 X ，若其上界函数值 $bp < L$ ，则可以断定 X 子树上不含最优答案结点，可以剪去以 X 为根的子树



华中农业大学信息学院



算法实现:

- 因为回溯求解的规则是"后进先出", 所以要用到栈来存储符合条件的解, 在存储过程中, 利用数组来存储各个物品的体积, 然后用深度优先的搜索方式求解, 将符合条件的数组元素的下标存入栈里, 最后得到符合条件的解并且实现输出。

华中农业大学信息学院



回溯法解决0/1背包问题

- `#include<stdio.h>`
- `int c; //背包容量`
- `int n; //物品数`
- `int weight[100]; //存放n个物品重量的数组`
- `int price[100]; //存放n个物品价值的数组`
- `int currentWeight=0; //当前重量`
- `int currentPrice=0; //当前价值`
- `int bestPrice=0; //当前最优值`
- `int bestAnswer[100]; //当前最优解`
- `int bp=0;`
- `int bA[100]; //当前最优解`
- `int times=0;`

华中农业大学信息学院



回溯法解决01背包问题

```
void Print();  
void Backtracking(int i)  
{  
    times+=1;  
    if(i>n)  
    {  
        Print();  
        if(bestPrice>bp)  
        {  
            bp=bestPrice;  
            for(int j=1;j<=n;j++)  
                bA[j]=bestAnswer[j];  
        }  
    }  
    return;  
}
```

华中农业大学信息学院



回溯法解决01背包问题

```
if(currentWeight+weight[i]<=c)
{ //将物品i放入背包，搜索左子树
    bestAnswer[i] = 1;
    currentWeight += weight[i];
    bestPrice += price[i];
    Backtracking(i+1); //完成上面的递归，返回到上一结点，物品i不放入背包，
    准备递归右子树
    currentWeight -= weight[i];
    bestPrice -= price[i];
} bestAnswer[i] = 0;
Backtracking(i+1);
}
```

华中农业大学信息学院



回溯法解决01背包问题

```
void Print()
{
    int i;
    printf("\n路径为  {");
    for(i=1;i<n;++i)
        printf("%d,",bestAnswer[i]);
    printf("%d}\t价值为%d\n",bestAnswer[i],bestPrice);
}

void main()
{
    int i;
    /*输入部分*/
    printf("请输入物品的数量:\n");
    scanf("%d",&n);
    printf("请输入背包的容量(能承受的重量):\n");
    scanf("%d",&c);
    printf("请依次输入%d个物品的重量:\n",n);
}
```

华中农业大学信息学院



回溯法解决01背包问题

```
for(i=1;i<=n;i++)
    scanf("%d",&weight[i]);
printf("请依次输入%d个物品的价值:\n",n);
for(i=1;i<=n;i++)
    scanf("%d",&price[i]);
printf("各符合条件的路径为: \n");
Backtracking(1);
printf("*****\n");
printf("\nthe best answer is {");
for(i=1;i<n;++i)
    printf("%d,",bA[i]);
printf("%d}\tthe price is %d\n",bA[i],bp);
printf("\n\n总共搜索结点数%d\n",times);
}
```

华中农业大学信息学院



回溯法解决01背包问题

```
GA "C:\Documents and Settings\Administrator\桌面\cdd\Debug\Cpp6.exe"
请输入物品的数量:
4
请输入背包的容量<能承受的重量>:
10
请依次输入4个物品的重量:
2 3 4 5
请依次输入4个物品的价值:
42 12 40 25
各符合条件的路径为:
路径为 <1,1,0,0>      价值为54
路径为 <1,0,0,0>      价值为42
路径为 <0,1,1,0>      价值为52
路径为 <0,1,0,1>      价值为37
路径为 <0,1,0,0>      价值为12
路径为 <0,0,1,1>      价值为65
路径为 <0,0,1,0>      价值为40
路径为 <0,0,0,1>      价值为25
路径为 <0,0,0,0>      价值为0
*****
the best answer is <0,0,1,1>      the price is 65

总共搜索结点数22
Press any key to continue_
```

华中农业大学信息学院



课堂练习

- 运动员最佳匹配问题。羽毛球队有男女运动员各 n 人。给定两个 $n \times n$ 矩阵 P 和 Q 。 $P[i][j]$ 是男运动员 i 和女运动员 j 配对组成混合双打的男运动员竞赛优势； $Q[i][j]$ 是女运动员 i 和男运动员 j 配合时女运动员的竞赛优势。由于技术配合和心理状态等各种因素影响， $P[i][j]$ 不一定等于 $Q[i][j]$ 。男运动员 i 和女运动员 j 配对组成混合双打的男女双方竞赛优势为 $P[i][j] * Q[j][i]$ 。设计一个算法，对于给定的男女运动员竞赛优势，计算男女运动员最佳配对法，使得各组男女双方竞赛优势的总和达到最大。

华中农业大学信息学院